UCS503P - Software Engineering Project

Faculty Office-Hours Scheduler

A Conflict-Free Booking System for Faculty and TA Office Hours

Submitted by:

1024170227 Manan Garg
1024270355 Vishesh
1024170240 Aryan
1024170244 Dravinam

Group: XXXX BE Third Year, CSE

Submitted to:

Dr. Raghav B. Venkataramaiyer

Prototype-Stage Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **5**
1.1 Background and Context 5
1.1.1 Problem Statement 5
1.2 Project Objectives 5

2 Methodology and Design **7**
2.1 System Architecture 7
2.1.1 High-Level Design 7
2.1.2 Technical Stack 7
2.2 Implementation Details 8
2.2.1 Module 1: Authentication and Role Management 8
2.2.2 Module 2: Conflict-Free Booking Logic 8
2.2.3 Module 3: Slot Publishing and Analytics 9

3 Results and Evaluation **13**
3.1 Testing Methodology 13
3.2 Performance Metrics 13
3.3 Analysis 14

4 Conclusion **15**
4.1 Summary of Work 15
4.2 Key Findings 15
4.3 Future Work and Recommendations 15
4.4 Final Remarks 16

Bibliography **17**

Chapter 1

Introduction

1.1 Background and Context

Students at Thapar routinely need short, one-on-one time with professors or teaching assistants to clear doubts on coursework. In practice, this coordination is entirely informal: students learn of a faculty member's availability by word of mouth, wait outside offices hoping to catch a free moment, or exchange emails and WhatsApp messages to arrange a meeting. There is no shared, structured record of who is free, when, or who has already booked that time.

This informality creates avoidable friction for both sides. Multiple students may show up expecting the same informal slot, while other free time on a faculty member's schedule goes completely unused. Since there is no record of what was agreed, there is no accountability for missed sessions – a student who does not show up leaves no trace, and a faculty member has no way to measure how often this happens.

1.1.1 Problem Statement

There is no lightweight, structured system through which faculty and TAs can publish their available time slots and have students reserve them without risk of double-booking or informal miscommunication. This project addresses that gap directly: it does not attempt to replace scheduling infrastructure at the institutional level, but provides a minimal, deployable booking layer that any faculty member or TA can adopt voluntarily. This report was prepared using LaTeX [Tea24].

1.2 Project Objectives

1. **Objective 1:** Allow faculty and TAs to publish available office-hour slots through a simple web interface, without needing any change to institutional systems.

2. **Objective 2:** Allow students to browse and book an open slot with a single action, with the system guaranteeing that no two students can ever book the same slot, even if they attempt to do so at the same instant.

3. **Objective 3:** Track booking outcomes (completed or no-show) so that no-show rate, slot utilization, and average wait time before a slot is available can be measured objectively from logged data, rather than estimated.

These objectives are specific (tied to concrete actions: publish, book, track), measurable (each has a corresponding numeric metric reported in Chapter 3), achievable within the tools and skills available to the team, relevant to a real and recurring problem on campus, and time-bound to the semester-long scope of this course.

Chapter 2

Methodology and Design

2.1 System Architecture

The system follows a standard three-tier web architecture: a browser-based frontend, a REST API backend, and a relational database. This separation allows the booking logic (the part of the system requiring the most care, see Section 2.2.2) to be implemented and tested independently of the user interface.

2.1.1 High-Level Design

Three actors interact with the system: **Student**, **Faculty/TA**, and **Admin**. Figure 2.1 shows the use case diagram capturing each actor's interactions with the system.

The system's core entities and their relationships are shown in the class diagram in Figure 2.2. `User` is a base type from which `Student`, `Faculty`, and `Admin` inherit, reflecting the role-based access control implemented in the backend. A `Faculty` publishes many `Slots`; a `Student` makes many `Bookings`; and each `Slot` is reserved by at most one `Booking` – this 0..1 cardinality is precisely what the database uniqueness constraint described in Section 2.2.2 enforces.

Figure 2.3 traces the control flow through the booking process, including the availability check that prevents double-booking.

2.1.2 Technical Stack

- **Framework/Language:** Node.js with Express for the backend REST API; plain HTML/JS for the prototype frontend (React planned for the final submission).

- **Database:** SQLite for the prototype stage, chosen for zero-configuration local development; PostgreSQL planned for the deployed final version.

- **Tools:** Node's built-in test runner (`node:test`) for backend unit tests; Git and GitHub for version control; GitHub Actions planned for CI/CD in the final stage.

- **Libraries:** `better-sqlite3` for synchronous, transactional database access; `jsonwebtoken` for stateless authentication; `bcryptjs` for password hashing.

2.2 Implementation Details

2.2.1 Module 1: Authentication and Role Management

Users sign up with a name, email, password, and role (`student`, `faculty`, or `admin`). Passwords are hashed with bcrypt before storage. On successful login, the backend issues a signed JSON Web Token (JWT) encoding the user's ID, role, and name. Every subsequent API request carries this token, and a middleware layer verifies it and enforces role-based access – for example, only a user with role `faculty` may call the slot-publishing endpoint.

2.2.2 Module 2: Conflict-Free Booking Logic

This is the most technically significant part of the system, and the part we tested most rigorously. The risk is a *race condition*: if two students attempt to book the same slot at nearly the same instant, a naive implementation (check availability, then separately write the booking) could let both succeed, since the check for student B might run before student A's write completes.

We address this with two mechanisms working together:

1. A `UNIQUE` constraint on the `slot_id` column of the `bookings` table at the database level, so the database itself physically cannot store two bookings for the same slot, regardless of application logic.

2. The check-and-reserve operation (verify the slot is open, then mark it booked, then insert the booking record) is wrapped in a single atomic database transaction, so no other operation can interleave between the check and the write.

Listing 2.4 shows the core of this implementation.

```
const bookSlot = db.transaction((slotId, studentId) => {
  const slot = db.prepare(
    "SELECT * FROM slots WHERE id = ? AND status = 'open'"
  ).get(slotId);
  if (!slot) throw new Error('SLOT_UNAVAILABLE');

  db.prepare(
    "UPDATE slots SET status='booked' WHERE id=? AND status='open'"
  ).run(slotId);

  return db.prepare(
    'INSERT INTO bookings (slot_id, student_id) VALUES (?,?)'
  ).run(slotId, studentId);
});
```

Listing 2.4: Transactional check-and-reserve booking logic

2.2.3 Module 3: Slot Publishing and Analytics

Faculty publish slots by submitting a timestamp; the system rejects duplicate slots for the same faculty member at the same time via a second uniqueness constraint on `(faculty_id, slot_time)`. An analytics endpoint, restricted to the `admin` role, aggregates booking outcomes into the metrics defined in our proposal: total bookings, no-show rate, and slot utilization rate.

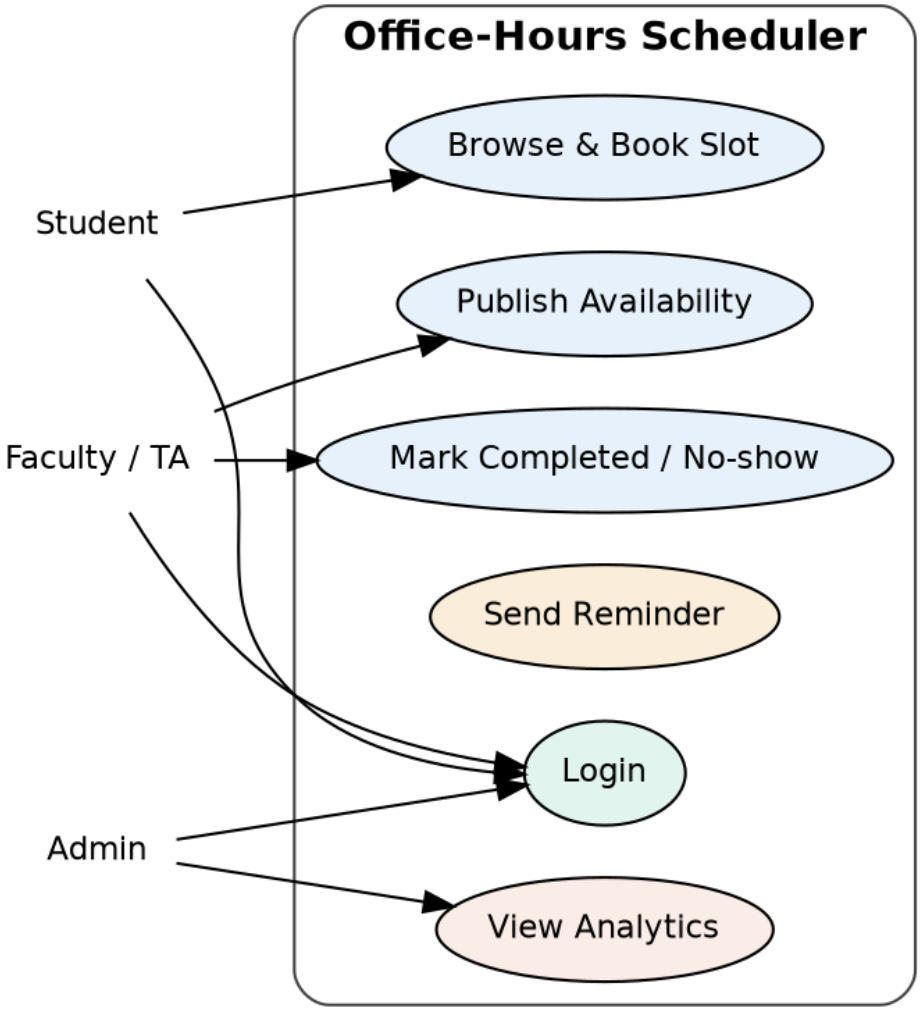


Figure 2.1: Use case diagram for the Office-Hours Scheduler

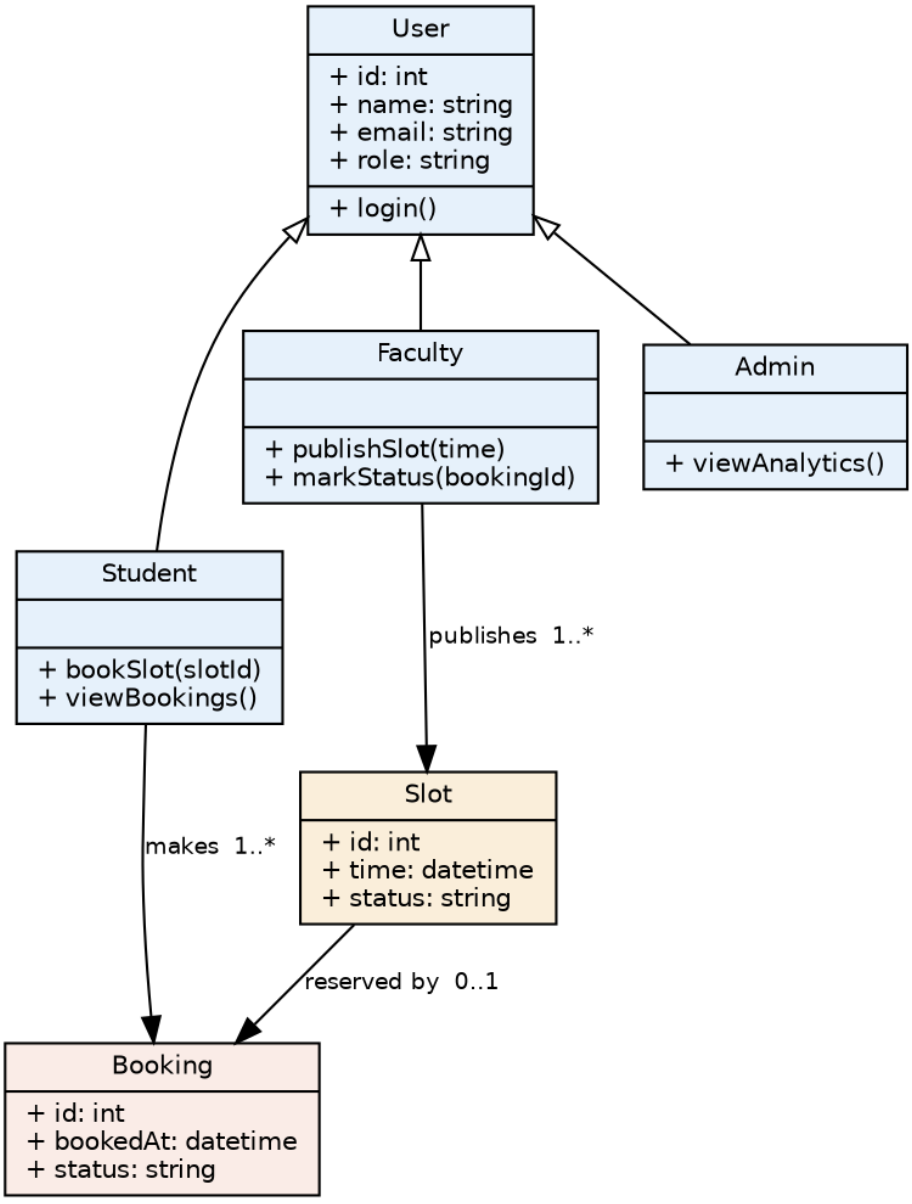
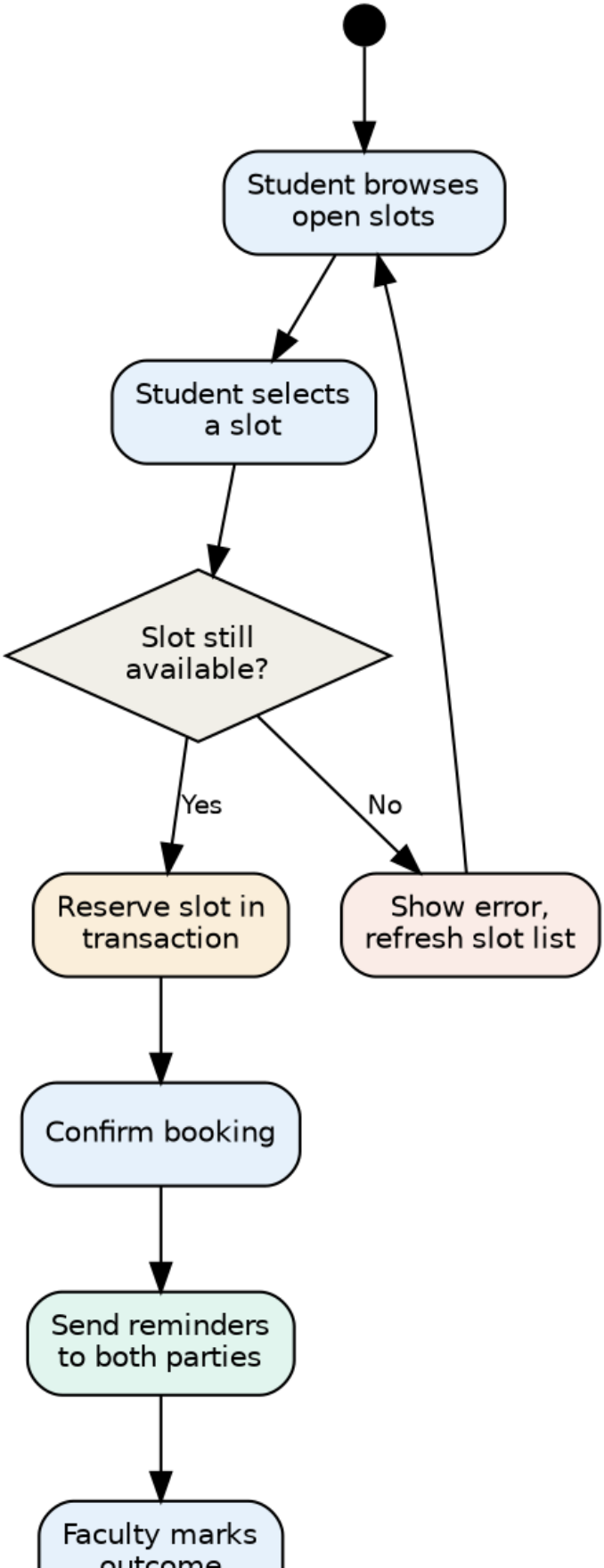


Figure 2.2: Class diagram showing core entities and relationships


Student browses open slots
Student selects a slot
Slot still available?
Yes
No
Reserve slot in transaction
Show error, refresh slot list
Confirm booking
Send reminders to both parties
Faculty marks

Chapter 3

Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** The booking transaction logic (Section 2.2.2) is covered by automated tests using Node's built-in test runner, run against an in-memory SQLite database for speed and isolation.

- **Integration Testing:** The full request flow – signup, login, slot publishing, booking, and conflict handling – was verified end-to-end via HTTP requests against a running instance of the backend.

- **Manual Testing:** The prototype frontend was used manually to sign up as a faculty member, publish a slot, sign up as two different students, and attempt to book the same slot from both accounts.

3.2 Performance Metrics

The central engineering claim of this project is that the booking system prevents double-booking under concurrent access. Table 3.1 reports the outcome of both the automated test suite and a manual two-student conflict test performed against the running prototype.

Test	**Expected**	**Observed**
Single booking on open slot	Success (200)	Success (200)
Second booking on same slot	Rejected (409)	Rejected (409)
Booking non-existent slot	Rejected	Rejected
Automated unit tests (2 cases)	2/2 pass	2/2 pass

Table 3.1: Booking conflict-prevention test results

In the manual two-student test, Student A's booking request for a published slot returned a confirmed booking. Student B's subsequent request for the identical slot was rejected with the message "This slot was just booked by someone else" and HTTP status 409, confirming that the uniqueness constraint and transactional logic behave as designed.

3.3 Analysis

At this prototype stage, we have met Objectives 1 and 2 in full: faculty can publish slots, and students can book them with a verified guarantee against double-booking. The core technical risk identified in our proposal – concurrent booking conflicts – has been directly tested and resolved, rather than assumed to work.

Objective 3 (measurable no-show rate and utilization tracking) is partially implemented: the analytics endpoint computes these metrics correctly from logged data, but we have not yet run a real pilot with actual faculty and students, so the numbers reported by the endpoint are currently based on test data rather than genuine usage. This is the primary gap to close before the final submission.

The main challenge encountered was deciding how to simulate concurrent requests in an automated test; we resolved this by testing the transaction's correctness directly (a booking attempt on an already-booked slot must raise an error) rather than attempting to physically race two HTTP requests against each other, which would introduce test flakiness without adding confidence beyond what the transactional guarantee already provides.

Chapter 4

Conclusion

4.1 Summary of Work

At the prototype stage, we have implemented and tested the core booking workflow of the Faculty Office-Hours Scheduler: authentication with role-based access, slot publishing by faculty, and conflict-free slot booking by students, backed by a database-level uniqueness constraint and an atomic transaction. This directly satisfies Objectives 1 and 2 from Chapter 1 and provides verified evidence, not just a design claim, that the system's central technical risk has been addressed.

4.2 Key Findings

- **Finding 1:** A database-level uniqueness constraint combined with a single atomic transaction is sufficient to prevent double-booking, without needing any application-level locking mechanism.
- **Finding 2:** Separating the booking transaction logic from the HTTP layer made it possible to unit-test the core correctness guarantee in isolation, independent of the web framework.
- **Finding 3:** SQLite was sufficient for rapid prototype iteration and testing; migrating to PostgreSQL for the final, deployed version should be a low-risk change since the transactional pattern used is standard SQL.

4.3 Future Work and Recommendations

1. Migrate the database layer from SQLite to PostgreSQL for the final submission, to match the production-oriented stack described in our proposal.
2. Rebuild the frontend in React, replacing the current minimal HTML/JS prototype, to support a more complete faculty availability console and admin dashboard.
3. Add email/in-app reminder notifications ahead of a booked session.
4. Run a real pilot with 3–5 willing faculty/TAs and their students to populate the analytics endpoint with genuine usage data, and measure actual no-show rate and slot utilization against the targets set in our proposal.

5. Set up the GitHub Actions CI/CD pipeline to run the existing test suite automatically on every push, and deploy the backend to a staging environment (Render or Railway).

4.4 Final Remarks

Building and testing the booking logic first, before any interface polish, let us verify the project's central engineering claim early and with confidence. The remaining work is primarily about breadth (a fuller frontend, real deployment, a genuine pilot) rather than about resolving open technical risk.

Bibliography

[Tea24] LaTeX Project Team. *LaTeX2e: An updated version of LaTeX*. 2024 (cit. on p. 5).